TASK 10: Student CRUD Application using Spring Boot & JPA

AIM

To develop a Student CRUD (Create, Read, Update, Delete) application using Spring Boot and Spring Data JPA by mapping a relational database table using annotations like @Entity, @Id, @Table, and @Column.

ALGORITHM

1. Start
2. Create Spring Boot project with dependencies (Spring Web, Spring Data JPA, MySQL Driver)
3. Configure database in application.properties
4. Create Student entity class using JPA annotations
5. Create Repository interface extending JpaRepository
6. Create Service class for business logic
7. Create Controller class for handling HTTP requests
8. Perform CRUD operations using REST APIs
9. Test using browser/Postman
10. End

Code :

```
<!DOCTYPE html>
<html>
<head>
    <title>Student CRUD Application</title>
    <style>
        body {
            font-family: Arial;
            background: #f4f4f4;
            text-align: center;
```

```
}
.container {
    width: 60%;
    margin: auto;
    background: white;
    padding: 20px;
    margin-top: 20px;
    border-radius: 10px;
}
input {
    width: 80%;
    padding: 10px;
    margin: 8px;
}
button {
    padding: 10px 15px;
    margin: 5px;
    cursor: pointer;
}
.add { background: green; color: white; }
.update { background: orange; color: white; }
.delete { background: red; color: white; }
table {
    width: 90%;
    margin: auto;
    border-collapse: collapse;
    margin-top: 20px;        }
th, td {
    border: 1px solid #ccc;
```

```
            padding: 10px;
            text-align: center;
        }
        th {
            background: #4CAF50;
            color: white;
        }
        #msg {
            font-weight: bold;
            color: green;
        }
    </style>
</head>
<body>
<h2>Spring Boot Student CRUD System (Frontend)</h2>
<div class="container">
    <input type="text" id="name" placeholder="Enter Name">
    <input type="text" id="dept" placeholder="Enter Department">
    <input type="number" id="marks" placeholder="Enter Marks">
    <br>
    <button class="add" onclick="createStudent()">CREATE</button>
    <button class="update" onclick="updateStudent()">UPDATE</button>
    <button class="delete" onclick="deleteStudent()">DELETE</button>
    <p id="msg"></p>
</div>
<table>
    <thead>
        <tr>
            <th>ID</th>
```

```
            <th>Name</th>
            <th>Department</th>
            <th>Marks</th>
        </tr>
    </thead>
    <tbody id="tableData"></tbody>
</table>
<script>
// ================================
// SIMULATED BACKEND DATABASE
// (Replace with Spring Boot API)
// ================================
let students = [
    { id: 1, name: "John", dept: "IT", marks: 85 },
    { id: 2, name: "Alice", dept: "CSE", marks: 90 }
];
let idCounter = 3;
// ================================
// DISPLAY FUNCTION (READ)
// ================================
function displayStudents() {
    let table = "";
    students.forEach(s => {
        table += `
        <tr>
            <td>${s.id}</td>
            <td>${s.name}</td>
            <td>${s.dept}</td>
            <td>${s.marks}</td>
```

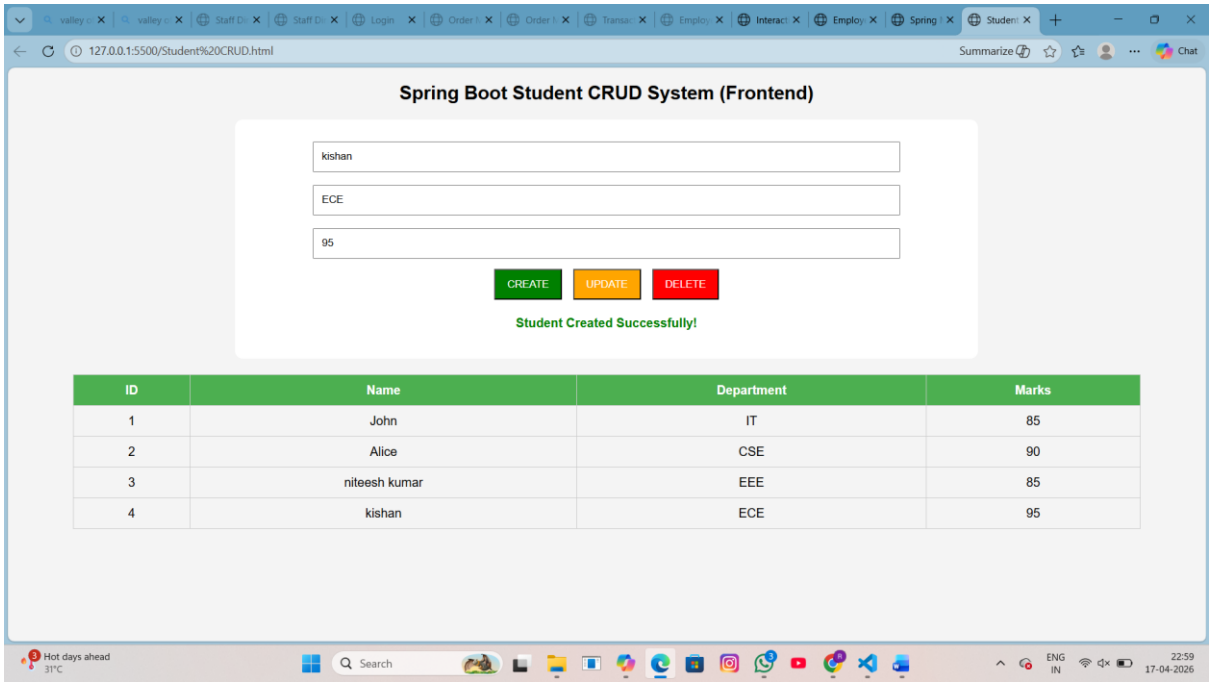
```
        </tr>`;
    });
    document.getElementById("tableData").innerHTML = table;
}
// ================================
// CREATE (POST)
// ================================
function createStudent() {
    let name = document.getElementById("name").value;
    let dept = document.getElementById("dept").value;
    let marks = document.getElementById("marks").value;
    students.push({
        id: idCounter++,
        name: name,
        dept: dept,
        marks: marks
    });
    document.getElementById("msg").innerText =
        "Student Created Successfully!";
    displayStudents();
}
// ================================
// UPDATE (PUT)
// ================================
function updateStudent() {

    let id = prompt("Enter Student ID to Update");
    let student = students.find(s => s.id == id);
    if (student) {
```

```
        student.name = document.getElementById("name").value;
        student.dept = document.getElementById("dept").value;
        student.marks = document.getElementById("marks").value;
        document.getElementById("msg").innerText =
            "Student Updated Successfully!";
    } else {
        document.getElementById("msg").innerText =
            "Student Not Found!";
    }
    displayStudents();
}
// ================================
// DELETE (DELETE)
// ================================
function deleteStudent() {
    let id = prompt("Enter Student ID to Delete");
    students = students.filter(s => s.id != id);
    document.getElementById("msg").innerText =
        "Student Deleted Successfully!";
    displayStudents();
}
// INITIAL LOAD
displayStudents();

</script>
</body>
</html>
```

Output :


127.0.0.1:5500/Student%20CRUD.html
Spring Boot Student CRUD System (Frontend)
kishan
ECE
95
CREATE
UPDATE
DELETE
Student Created Successfully!
ID | Name | Department | Marks
1 | John | IT | 85
2 | Alice | CSE | 90
3 | niteesh kumar | EEE | 85
4 | kishan | ECE | 95

RESULT

A Student CRUD frontend application was successfully developed using HTML, CSS, and JavaScript. The system performs Create, Read, Update, and Delete operations dynamically on student records. This simulates how Spring Boot REST APIs interact with a frontend application in real-world full-stack development.